Εθνικό Μετσόβιο Πολυτεχνείο
Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών
Τομέας Τεχνολογίας Πληροφορικής και Υπολογιστών
Μάθημα: ″Σχεδιασμός Ενσωματωμένων Συστημάτων″
Ακαδ. Έτος: 2026-2027

1η Εργαστηριακή Άσκηση

Βελτιστοποίηση Αλγορίθμων για Χαμηλή Κατανάλωση Ενέργειας και Υψηλή Απόδοση

Αποστολόπουλος Νικόλαος
(03122614)
Γκριμπογιάννης Χαρίλαος
(03122002)

Περίληψη

Η παρούσα εργασία υλοποιήθηκε στα πλαίσια του μαθήματος ″Σχεδιασμός Ενσωματωμένων Συστημάτων″ και αφορά ...

Περιεχόμενα

1 Βελτιστοποίηση της Εφαρμογής tables.c **3**
1.1 Σύγκριση Αρχιτεκτονικών 3
1.2 Loop Unrolling 4
1.3 Μέγεθος L1 Data Cache 5
1.4 Βελτιστοποίηση με Εξαντλητική Αναζήτηση 5
1.5 Βελτιστοποίηση με τη χρήση Γενετικού Αλγορίθμου 7

1 Βελτιστοποίηση της Εφαρμογής tables.c

Στην παρούσα παράγραφο βελτιστοποιούμε την εφαρμογή tables.c (βλ. Listing 1) εκτελούμενη σε δύο διαφορετικές αρχιτεκτονικές x86 με χρήση του προσομοιωτή Gem5. Η πρώτη αρχιτεκτονική που μελετάμε αφορά μια αρχιτεκτονική επεξεργαστή και κύριας μνήμης, ενώ η δεύτερη είναι επαυξημένη με L1 Instruction Cache, L1 Data Cache και L2 Cache. Συγκεκριμένα, μελετάμε την επίδραση της τεχνικής του loop unrolling, καθώς και την επίδραση της ύπαρξης κρυφών μνημών διαφορετικών μεγεθών στον χρόνο εκτέλεσης της εφαρμογής μετρημένο σε κύκλους ρολογιού.

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int i;
    int N = 100000;
    double *x1, *x2, *x3, *y;
    double a1 = 0.5;
    double a2 = 1;
    double a3 = 1.5;

    x1 = (double*) malloc(N * sizeof(double));
    x2 = (double*) malloc (N * sizeof(double));
    x3 = (double*) malloc (N * sizeof(double));
    y = (double*) malloc (N * sizeof(double));

    //Do not modify this loop
    for (i=0; i<=N-1; i++)
    {
        x1[i] = (double) i * 0.5;
        x2[i] = (double) i * 0.8;
        x3[i] = (double) i * 0.2;
        y[i] = 0;
    }

    for (i=0; i<=N-1; i++)
    {
        y[i] = y[i] + a1*x1[i] + a2*x2[i] + a3*x3[i];
    }

    return 0;

}
```

Listing 1: Source Code (tables.c)

1.1 Σύγκριση Αρχιτεκτονικών

Αρχικά προσομοιώνουμε την εκτέλεση της εφαρμογής tables.exe σε αρχιτεκτονική που περιλαμβάνει τον επεξεργαστή και την κύρια μνήμη. Η εκτέλεση γίνεται σε 858,490,254 κύκλους. Ύστερα προσομοιώνουμε την εκτέλεση της ίδιας εφαρμογής σε σύστημα που διαθέτει επιπλέον L1 Instruction και L1 Data Caches μεγέθους 8kB και L2 Cache μεγέθους 128kB. Η εφαρμογεί εκτελείται σε 59,829,295 κύκλους. Παρατηρούμε ότι η ενσωμάτωση κρυφών μνημών στο σύστημα μείωσε τον χρόνο εκτέλεσης κατά 798,660,959 κύκλους δίνοντας ένα speedup περίπου 14.35x. Η δραματική αυτή αύξηση της ταχύτητας εκτέλεσης αποδεικνύει έμπρακτα την ανάγκη ύπαρξης των κρυφών μνημών, καθώς χωρίς αυτές

κάθε ανάγνωση/εγγραφή που εκτελεί το πρόγραμμα απαιτεί πολύ αργές προσπελάσεις στην κύρια μνήμη.

1.2 Loop Unrolling

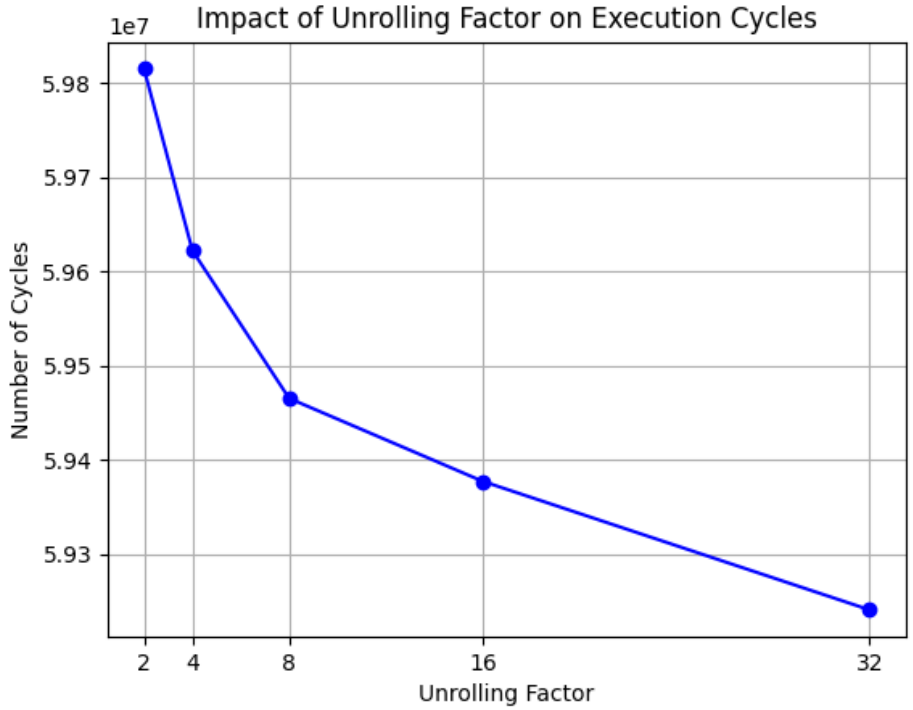
Αφού αποδείξαμε την ανωτερότητα της αρχιτεκτονικής με κρυφές μνήμες, θα μελετήσουμε πλέον μόνο αυτή την αρχιτεκτονική. Για να ελαττώσουμε περαιτέρω τον χρόνο εκτέλεσης της εφαρμογής, εισάγουμε τη μέθοδο loop unrolling και εξετάζουμε την επίδραση της εφαρμογής της.

Η μέθοδος loop unrolling είναι ένας μετασχηματισμός που συνίσταται στο "ξετύλιγμα" του βρόχου. Στο παρόν την εφαρμόζουμε στον βρόχο υπολογισμού του πίνακα y όπως φαίνεται στο Listing 2.

```
for (i = 0; i <= N - k; i = i + k) {
    y[i] = y[i] + a1 * x1[i] + a2 * x2[i] + a3 * x3[i];
    ...
    y[(i + k)] = y[(i + k)] + a1 * x1[(i + k)] + a2 * x2[(i + k)] + a3 * x3[(i +
    k)];
}
for (int i = N - ((N - (0)) % k); i <= N - 1; i = i + 1)
    y[i] = y[i] + a1 * x1[i] + a2 * x2[i] + a3 * x3[i];
```

Listing 2: Loop Unrolling with Unrolling Factor k

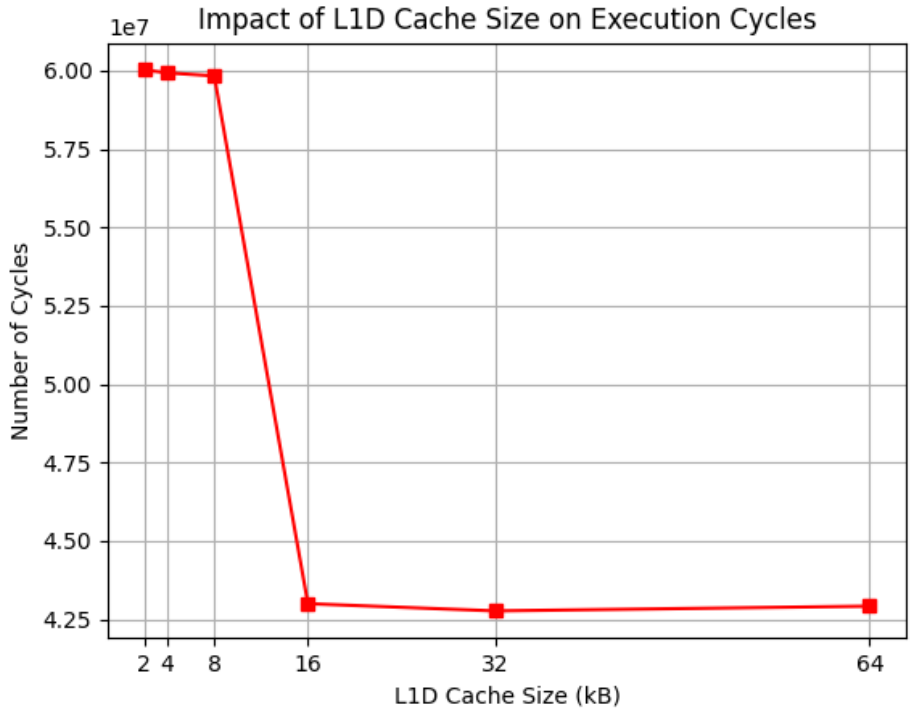
Στο Σχήμα 1 φαίνεται η επίδραση του μετασχηματισμού στον χρόνο εκτέλεσης σε πλήθος κύκλων ρολογιού για διαφορετικά unrolling factors (2, 4, 8, 16 και 32). Παρατηρούμε ότι ... Αυτό οφείλεται...



Σχήμα 1: Επίδραση του Unrolling Factor στον Χρόνο Εκτέλεσης

1.3 Μέγεθος L1 Data Cache

Συνεχίζοντας τη μελέτη της δεύτερης αρχιτεκτονικής, προχωράμε σε μια εξερεύνηση της επίδρασης διαφορετικών μεγεθών L1 Data Cache (2kB, 4 kB, 8kB, 16kB, 32kB και 64kB).



Σχήμα 2: Επίδραση του Μεγέθους της L1 Data Cache στον Χρόνο Εκτέλεσης

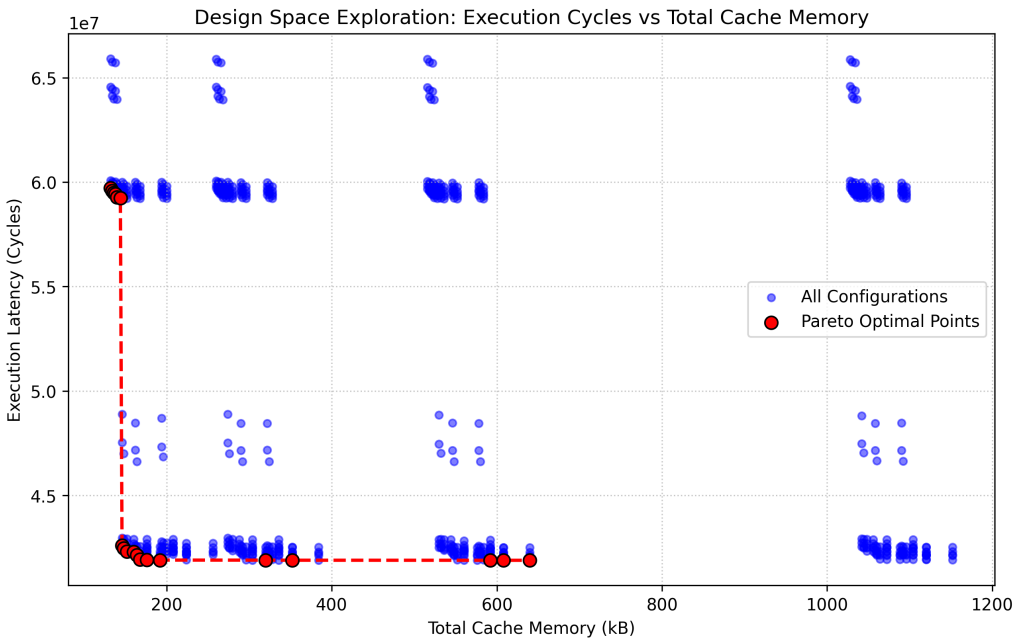
Εδώ παρατηρούμε κάτι πολύ ενδιαφέρον...

1.4 Βελτιστοποίηση με Εξαντλητική Αναζήτηση

Έχοντας μελετήσει την επίδραση συγκεκριμένων παραμέτρων της εφαρμογής (Unrolling Factor) αλλά και της αρχιτεκτονικής (Μέγεθος L1 Data Cache), έχουμε σχηματίσει μια εικόνα για τη βελτιστοποίηση. Αυτό που μας ενδιαφέρει είναι να ελαχιστοποιήσουμε το πλήθος των κύκλων που απαιτούνται για την εκτέλεση της εφαρμογής αλλά και τη συνολική μνήμη που θα χρησιμοποιήσουμε (το άθροισμα των L1I, L1D και L2 σε kB). Έτσι, καταγράφουμε έναν χώρο προς εξερεύνηση, ο οποίος ορίζεται από τους συνδυασμούς των παρακάτω

L1D Cache Size $\in$ [2kB, 4kB, 8kB, 16kB, 32kB, 64kB]
L1I Cache Size $\in$ [2kB, 4kB, 8kB, 16kB, 32kB, 64kB]
L2 Cache Size $\in$ [128kB, 256kB, 512kB, 1024kB]
Unrolling Factor $\in$ [2, 4, 8, 16, 32]

Προχωράμε σε εξαντλητική αναζήτηση του χώρου εξερεύνησης και για κάθε ένα από τα 720 σημεία του χώρου καταγράφουμε το πλήθος των κύκλων που απαιτούνται για την εκτέλεση της εφαρμογής. Στο Σχήμα 3 φαίνονται οι θέσεις όλων των σημείων και το Pareto Frontier, ενώ στον Πίνακα 1 παραθέτουμε το Pareto Frontier μαζί με τα στοιχεία της αρχιτεκτονικής που οδήγησαν στο εκάστοτε Pareto Optimal σημείο.



Σχήμα 3: Exhaustive Search Pareto Frontier

L1I (kB)	L1D (kB)	L2 (kB)	UF	Total Memory (kB)	Latency (CC)
2	2	128	8	132	59724828
2	16	128	8	146	42591707
4	2	128	16	134	59579732
4	4	128	16	136	59484646
4	16	128	16	148	42472280
4	32	128	16	164	42146735
8	2	128	32	138	59427801
8	4	128	32	140	59281539
8	8	128	32	144	59241019
8	16	128	32	152	42315832
8	32	128	32	168	41928215
16	16	128	32	160	42300132
16	32	128	32	176	41912388
16	64	512	32	592	41889209
32	32	128	32	192	41898540
32	32	256	32	320	41896021
32	64	256	32	352	41892206
32	64	512	32	608	41889106
64	64	512	32	640	41888238

Πίνακας 1: Exhaustive Search Pareto Optimal Points

Παρατηρούμε ότι... Τέλος, σημειώνουμε ότι λόγω του μεγάλου πλήθους προσομοιώσεων (720 σημεία), για την εξαντλητική αναζήτηση απαιτήθηκαν 2 ώρες και 46 λεπτά.

1.5 Βελτιστοποίηση με τη χρήση Γενετικού Αλγορίθμου

Αντί για τη χρονοβόρα εξαντλητική αναζήτηση του χώρου εξερεύνησης, μπορούμε να χρησιμοποιήσουμε έναν γενετικό αλγόριθμο για να λάβουμε το Pareto Frontier. Ο αλγόριθμος αυτός μπλα μπλα μπλα...

Εκτελώντας τον αλγόριθμο παρατηρούμε ότι κάνει μόνο 30 evaluations και τερματίζει σε 7 λεπτά και 30 δευτερόλεπτα, επιστρέφοντας το Pareto Frontier του Πίνακα 2. Παρατηρούμε αρχικά ότι ο αλγόριθμος βρήκε 3 Pareto Optimal σημεία, σε αντίθεση με τα 19 της εξαντλητικής αναζήτησης. Τέλος, παρατηρούμε ότι μόνο ένα εκ των τριών σημείων (32kB, 32kB, 256kB, 32) είναι στην πραγματικότητα Pareto Optimal και βρίσκεται στα αποτελέσματα της προηγούμενης υποπαραγράφου. Αυτό συμβαίνει καθώς ο αλγόριθμος βασίζεται σε έναν ευρετικό (heuristic) μηχανισμό...

L1I (kB)	L1D (kB)	L2 (kB)	UF	Total Memory (kB)	Latency (CC)
32kB	32kB	512kB	32	576	41895278
16kB	32kB	128kB	16	176	42130415
32kB	32kB	256kB	32	320	41896021

Πίνακας 2: Genetic Algorithm Pareto Optimal Points